

Model solution and feedback for G51DBS coursework 1: Entity-relationship diagram.

Several different valid solutions were possible. My solution (and that of the majority of students) is below. I gave some variations on this solution and some common mistakes in italics, please ignore them at first reading if it looks confusing.

1.

Entities: Patient, Ward, Nurse, Doctor.

(Some valid variations: add Hospital as an Entity; make Ward an attribute of Patient, Nurse and Doctor, rather than an entity; make Nurses and Doctors into one entity Staff, which has an attribute Job with values Doctor or Nurse).

Attributes: Patients have Admission Number, Name, Address, Telephone Number. Wards have a Name (which can be A&E, Cardiology, Oncology, etc.).

(Some valid variations: although it is not in the text of the question, it is perfectly all right to assume that Nurses and Doctors have an attribute Name etc. It is also OK to assume that Patients have both a permanent ID and an admission number which changes between admissions).

Common mistakes:

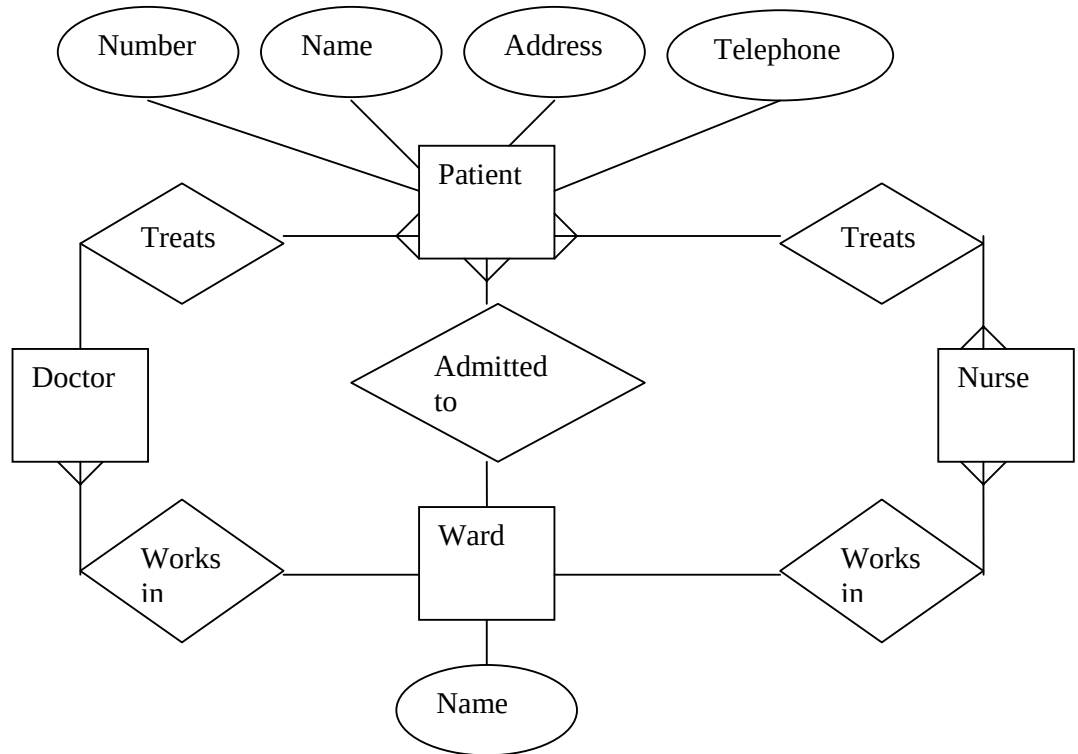
- a) *Saying that Wards have attributes A&E, Cardiology, Oncology: these are not attributes but values of an attribute Name of the Ward.*
- b) *Adding attributes such as WardID to the Patient: these are not really attributes of Patients; the information about patient's ward in the entity-relationship model is encoded in the relationship between Patients and Wards. (In the SQL tables, relationships are turned into foreign keys, so there will be a column name WardID in the Patient table.)*

Relationships: Patients are Admitted to Wards; Nurses are Working in a Ward; Doctors are Working in a Ward; Doctors Treat Patients; Nurses Treat Patients.

Cardinality ratios:

- Wards and Patients: 1:M
- Wards and Nurses: 1:M
- Wards and Doctors: 1:M
- Doctors and Patients: 1:M
- Nurses and Patients: M:M *(another common mistake was to say it is 1:M)*

2. Diagram for the list of entities, attributes and relationships given above.



Note that the relationship between Patients and Nurses is M:M in the diagram. It is obviously OK to create a new entity such as NurseRoster to make it 1:M between Nurse and NurseRoster and between Patient and NurseRoster, but not necessary given the question.

3. SQL tables. Note that relationships have turned into foreign keys, and I have added one more table to model the relationship between Patients and Nurses (NurseRoster). Obviously precise column names, constraint names, and exact data types do not matter.

```

CREATE TABLE Ward (
    wardName VARCHAR(50) NOT NULL,
    CONSTRAINT pk_ward PRIMARY KEY (wardName)
);
    
```

```

CREATE TABLE Nurse (
    nurseName VARCHAR(50) NOT NULL,
    wardName VARCHAR(50) NOT NULL,
    CONSTRAINT pk_nurse PRIMARY KEY (nurseName),
    CONSTRAINT fk_nurseWard FOREIGN KEY (wardName) REFERENCES Ward
(wardName)
    
```

);

```
CREATE TABLE Doctor (  
    doctorName VARCHAR(50) NOT NULL,  
    wardName VARCHAR(50) NOT NULL,  
    CONSTRAINT pk_doctor PRIMARY KEY (doctorName),  
    CONSTRAINT fk_doctorWard FOREIGN KEY (wardName) REFERENCES Ward  
(wardName)  
);
```

```
CREATE TABLE Patient(  
    admNumber INT NOT NULL,  
    patientName VARCHAR(50) NOT NULL,  
    patientAddress VARCHAR(150) NOT NULL,  
    patientTelephone VARCHAR(15) NOT NULL,  
    wardName VARCHAR(50) NOT NULL,  
    doctorName VARCHAR(50) NOT NULL,  
    CONSTRAINT pk_patient PRIMARY KEY (admNumber),  
    CONSTRAINT fk_patientWard FOREIGN KEY (wardName) REFERENCES Ward  
(wardName),  
    CONSTRAINT fk_patientDoctor FOREIGN KEY (doctorName) REFERENCES  
Doctor(doctorName)  
);
```

```
CREATE TABLE NurseRoster(  
    admNumber INT NOT NULL,  
    nurseName VARCHAR(50) NOT NULL,  
    CONSTRAINT pk_nurseRoster PRIMARY KEY (admNumber, nurseName),  
    CONSTRAINT fk_rosterPatient FOREIGN KEY (admNumber) REFERENCES  
Patient(admNumber),  
    CONSTRAINT fk_rosterNurse FOREIGN KEY (nurseName) REFERENCES Nurse  
(nurseName)  
);
```